



Universidad Libre De Colombia
Facultad De Ingeniería

Presenta:
Santiago Enrique Arquez Cervantes

Tutor:
Miguel Hernández Bejarano

Bogotá D.C.

Enunciado

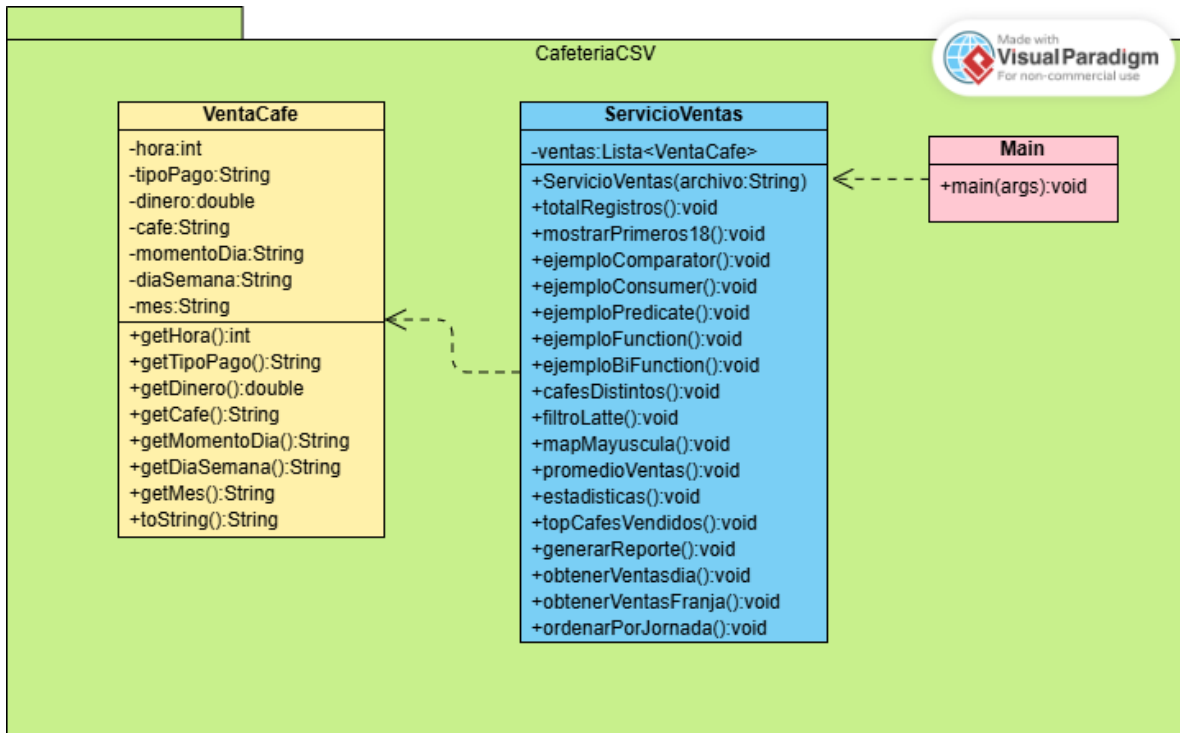
Una cafetería registra cada una de sus ventas en un archivo CSV (Coffe_sales.csv) con información sobre la hora, el tipo de pago, el valor de la venta, el café vendido, el momento del día, el día de la semana y el mes. Se requiere desarrollar un sistema en Java que lea este archivo y permita explorar y analizar la información utilizando la API de Streams y las interfaces funcionales del paquete `java.util.function` (`Comparator`, `Consumer`, `Predicate`, `Function` y `BiFunction`), generando además un reporte de ventas en un archivo de texto.

Requerimiento

El sistema debe cumplir con los siguientes requerimientos funcionales:

1. Leer el archivo CSV y construir, por cada línea, un objeto `VentaCafe` (omitiendo el encabezado).
2. Mostrar el total de registros cargados y los primeros 10 registros de la lista.
3. Ordenar las ventas por valor monetario usando `Comparator` (ejemplo `Comparator`) y por jornada del día (ordenarPorJornada).
4. Mostrar información resumida de una venta (café y valor) usando `Consumer` (ejemplo `Consumer`).
5. Filtrar ventas mayores a \$35 usando `Predicate` (ejemplo `Predicate`).
6. Construir una descripción de cada venta (café y valor) usando `Function` (ejemplo `Function`).
7. Combinar el nombre del café y su valor en un texto tipo reporte usando `BiFunction` (ejemplo `BiFunction`).
8. Listar los cafés distintos ofrecidos (`cafesDistintos`) y filtrar únicamente las ventas de Latte (`filtroLatte`).
9. Transformar los nombres de los cafés a mayúsculas usando `map` (`mapMayusculas`).
10. Calcular el promedio de ventas (`promedioVentas`) y estadísticas de total, máximo y mínimo (`estadísticas`).
11. Determinar los cafés más vendidos, agrupando y contando con `Collectors.groupingBy` (`topCafesVendidos`).
12. Generar un archivo de texto (`reporte_ventas.txt`) con el total de registros, el total vendido y el promedio (`generarReporte`).
13. Obtener las ventas realizadas en la mañana (`obtenerVentasdia`) y agrupar la cantidad de ventas por franja del día (`obtenerVentasFranja`).

Diagrama de clases



Código Java

Clase Main

```

1  import java.util.Scanner;
2  public class Main {
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          ServicioVentas servicio = null;
6          try {
7              servicio = new ServicioVentas(archivo: "data/Coffe_sales.csv");
8          }
9          catch (Exception e) {
10             System.out.println("Error al cargar el archivo: " + e.getMessage());
11             return;
12          }
13          int opcion;
14          do {
15              System.out.println(x: "¡Hola, encargado de ventas de café!");
16              System.out.println(x: "Ingrese su número de opción: ");
17              System.out.println(x: "1. Total de registros");
18              System.out.println(x: "2. Mostrar primeros 10 registros");
19              System.out.println(x: "3. Ejemplo Comparator (ordenar por dinero)");
20              System.out.println(x: "4. Ejemplo Consumer");
21              System.out.println(x: "5. Ejemplo Predicate (ventas > 35)");
22              System.out.println(x: "6. Ejemplo Function (descripción de venta)");
23              System.out.println(x: "7. Ejemplo BiFunction (reporte)");
24              System.out.println(x: "8. Cafés distintos");
25              System.out.println(x: "9. Filtrar solo Latte");
26              System.out.println(x: "10. Map a mayúsculas");
27              System.out.println(x: "11. Promedio de ventas");
28              System.out.println(x: "12. Estadísticas (total, máximo, mínimo)");
29              System.out.println(x: "13. Top cafés más vendidos");
30              System.out.println(x: "14. Generar reporte (archivo .txt)");
31              System.out.println(x: "15. Ventas en la mañana (Morning)");

```

```

32      System.out.println(x: "16. Ventas por franja horaria");
33      System.out.println(x: "17. Ordenar por jornada");
34      System.out.println(x: "0. Para salir del programa");
35      System.out.print(s: "Opción: ");
36      opcion = sc.nextInt();
37      sc.nextLine(); // Limpieza de buffer
38      try {
39          if (opcion == 1) {
40              servicio.totalRegistros();
41          }
42
43          else if (opcion == 2) {
44              servicio.mostrarPrimeros10();
45          }
46
47          else if (opcion == 3) {
48              servicio.ejemploComparator();
49          }
50
51          else if (opcion == 4) {
52              servicio.ejemploConsumer();
53          }

```

```

55          else if (opcion == 5) {
56              servicio.ejemploPredicate();
57          }
58
59          else if (opcion == 6) {
60              servicio.ejemploFunction();
61          }
62
63          else if (opcion == 7) {
64              servicio.ejemploBiFunction();
65          }
66
67          else if (opcion == 8) {
68              servicio.cafesDistintos();
69          }
70
71          else if (opcion == 9) {
72              servicio.filtroLatte();
73          }
74
75          else if (opcion == 10) {
76              servicio.mapMayusculas();
77          }
78
79          else if (opcion == 11) {
80              servicio.promedioVentas();
81          }
82

```

```

83         else if (opcion == 12) {
84             servicio.estadisticas();
85         }
86
87         else if (opcion == 13) {
88             servicio.topCafesVendidos();
89         }
90
91         else if (opcion == 14) {
92             servicio.generarReporte();
93         }
94
95         else if (opcion == 15) {
96             servicio.obtenerVentasdia();
97         }
98
99         else if (opcion == 16) {
100             servicio.obtenerVentasFranja();
101         }
102
103         else if (opcion == 17) {
104             servicio.ordenarPorJornada();
105         }
106
107         else if (opcion == 0) {
108             System.out.println(x: "¡Gracias por usar el sistema de ventas de café!");
109         }
110
111         else if (opcion == 0) {
112             System.out.println(x: "¡Gracias por usar el sistema de ventas de café!");
113         }
114
115         else {
116             System.out.println(x: "Opción no válida.");
117         }
118
119         catch (Exception e) {
120             System.out.println("Error: " + e.getMessage());
121         }
122
123     } while (opcion != 0);
124
125     sc.close();
126 }
127

```

Clase VentaCafe

```

cafeteriacsv > J VentaCafe.java > Java > VentaCafe > getDinero()
1 public class VentaCafe {
2     private int hora;
3     private String tipoPago;
4     private double dinero;
5     private String cafe;
6     private String momentoDia;
7     private String diaSemana;
8     private String mes;
9
10    public VentaCafe(int hora, String tipoPago, double dinero,
11                     String cafe, String momentoDia,
12                     String diaSemana, String mes) {
13        this.hora = hora;
14        this.tipoPago = tipoPago;
15        this.dinero = dinero;
16        this.cafe = cafe;
17        this.momentoDia = momentoDia;
18        this.diaSemana = diaSemana;
19        this.mes = mes; }
20
21    public int getHora() {
22        return hora;
23    }
24    public String getTipoPago() {
25        return tipoPago;
26    }
27    public double getDinero() {
28        return dinero;
29    }
30    public String getCafe() {
31        return cafe;
32    }
33    public String getMomentoDia() {
34        return momentoDia;
35    }
36    public String getDiaSemana() {
37        return diaSemana;
38    }
39    public String getMes() {
40        return mes;
41    }
42    @Override
43    public String toString() {
44        return String.format(
45            "Hora=%d | Cafe=%s | Venta=%.2f | Pago=%s | Momento=%s",
46            hora, cafe, dinero, tipoPago, momentoDia);
47    }
48 }

```

Clase ServicioVentas

```

1  import java.io.IOException;
2  import java.nio.file.Files;
3  import java.nio.file.Path;
4  import java.nio.file.Paths;
5
6  import java.util.Comparator;
7  import java.util.List;
8  import java.util.Map;
9  import java.util.function.BiFunction;
10 import java.util.function.Consumer;
11 import java.util.function.Function;
12 import java.util.function.Predicate;
13
14 import java.util.stream.Collectors;
15
16 public class ServicioVentas {
17     private List<VentaCafe> ventas;
18     public ServicioVentas(String archivo) throws IOException {
19         Path ruta = Paths.get(archivo);
20         ventas = Files.lines(ruta)
21             .skip(n: 1)
22             .map(linea -> linea.split(regex: ","))
23             .filter(datos -> datos.length >= 7)
24             .map(datos -> new VentaCafe(
25                 Integer.parseInt(datos[0]),
26                 datos[1],
27                 Double.parseDouble(datos[2]),
28                 datos[3],
29                 datos[4],
30                 datos[5],
31                 datos[6]
32             ))
33             .collect(Collectors.toList());
34     }
35     public void totalRegistros() {
36         System.out.println("\nTOTAL REGISTROS: " + ventas.size());
37     }
38     public void mostrarPrimeros10() {
39         System.out.println(x: "\nPRIMEROS 10 REGISTROS");
40         ventas.stream()
41             .limit(maxSize: 10)
42             .forEach(System.out::println);
43     }

```

```

44     public void ordenarPorJornada() {
45         System.out.println(x: "\nVENTAS ORDENADAS POR JORNADA");
46         ventas.stream()
47             .sorted(
48                 Comparator.comparing(
49                     VentaCafe::getMomentoDia
50                 )
51             )
52             .limit(maxSize: 10)
53             .forEach(System.out::println);
54     }
55     public void ejemploComparator() {
56         System.out.println(x: "\nCOMPARATOR");
57
58         ventas.stream()
59             .sorted(Comparator.comparing(VentaCafe::getDinero))
60             .limit(maxSize: 10)
61             .forEach(System.out::println);
62     }
63
64     public void ejemploConsumer() {
65         //Mostrar información completa de una venta.
66         //Mostrar únicamente café y valor vendido.
67
68         System.out.println(x: "\nCONSUMER");
69
70         Consumer<VentaCafe> consumidor =
71             venta -> System.out.println(
72                 venta.getCafe() +
73                 " -> $" +
74                 venta.getDinero()
75             );
76
77         ventas.stream()
78             .limit(maxSize: 10)
79             .forEach(consumidor);
80     }

```



```

80     }
81
82     public void ejemploPredicate() {
83         //Ventas mayores a 35.
84         //Ventas menores a 30.
85         //Cafés Latte.
86         //Cafés Cappuccino.
87         //Ventas realizadas en Morning.
88
89         System.out.println(x: "\nPREDICATE");
90
91         Predicate<VentaCafe> ventaMayor35 =
92             venta -> venta.getDinero() > 35;
93
94         ventas.stream()
95             .filter(ventaMayor35)
96             .limit(maxSize: 10)
97             .forEach(System.out::println);
98     }
99
100    public void ejemploFunction() {
101        //Convertir nombre del café a mayúsculas.
102        //Obtener únicamente el valor monetario.
103        //Crear una descripción personalizada de la venta.
104
105        System.out.println(x: "\nFUNCTION");
106
107        Function<VentaCafe,String> descripcion =
108            venta ->
109                venta.getCafe() +
110                " - $" +
111                venta.getDinero();
112
113        ventas.stream()
114            .limit(maxSize: 10)
115            .map(descripcion)
116            .forEach(System.out::println);
117    }
118
119    public void ejemploBiFunction() {
120        //Combinar nombre del café y valor monetario.
121        //Construir una descripción tipo reporte.
122
123        System.out.println(x: "\nBIFUNCTION");

```

```

125        BiFunction<String,Double,String> reporte =
126            (cafe, dinero) ->
127                "Producto: " + cafe +
128                " | Venta: $" + dinero;
129
130        ventas.stream()
131            .limit(maxSize: 10)
132            .forEach(v ->
133                System.out.println(
134                    reporte.apply(
135                        v.getCafe(),
136                        v.getDinero()
137                    )
138                ));
139    }
140    public void cafesDistintos() {
141        //Mostrar todos los cafés distintos.
142        //Calcular total de ventas.
143        //Contar registros.
144
145        System.out.println(x: "\nCAFES DISTINTOS");
146
147        ventas.stream()
148            .map(VentaCafe::getCafe)
149            .distinct()
150            .sorted()
151            .forEach(System.out::println);
152    }
153
154    public void filtroLatte() {
155
156        System.out.println(x: "\nSOLO LATTE");
157
158        ventas.stream()
159            .filter(v -> v.getCafe().equalsIgnoreCase(anotherString: "Latte"))
160            .limit(maxSize: 10)
161            .forEach(System.out::println);
162    }
163    public void mapMayusculas() {
164
165        System.out.println(x: "\nMAP");
166
167        ventas.stream()
168            .map(v -> v.getCafe().toUpperCase())

```

```

167     ventas.stream()
168         .map(v -> v.getCafe().toUpperCase())
169         .distinct()
170         .limit(maxSize: 10)
171         .forEach(System.out::println);
172 }
173
174 public void promedioVentas() {
175
176     double promedio =
177         ventas.stream()
178             .mapToDouble(VentaCafe::getDinero)
179             .average()
180             .orElse(0);
181
182     System.out.println("\nPROMEDIO: " + promedio);
183 }
184
185 public void estadisticas() {
186
187     double total =
188         ventas.stream()
189             .mapToDouble(VentaCafe::getDinero)
190             .sum();
191
192     double maximo =
193         ventas.stream()
194             .mapToDouble(VentaCafe::getDinero)
195             .max()
196             .orElse(0);
197
198     double minimo =
199         ventas.stream()
200             .mapToDouble(VentaCafe::getDinero)
201             .min()
202             .orElse(0);
203
204     System.out.println("\nTOTAL VENDIDO: " + total);
205     System.out.println("MAXIMO: " + maximo);
206     System.out.println("MINIMO: " + minimo);
207 }
208
209 public void topCafesVendidos() {
210
211     System.out.println(x: "\nTOP 10 CAFES");

```

```

212
213     Map<String, Long> resultado =
214         ventas.stream()
215             .collect(Collectors.groupingBy(
216                 VentaCafe::getCafe,
217                 Collectors.counting()));
218
219     resultado.entrySet()
220         .stream()
221         .sorted(Map.Entry.<String, Long>
222             comparingByValue()
223             .reversed())
224         .limit(maxSize: 10)
225         .forEach(System.out::println);
226 }
227
228 public void generarReporte() throws IOException {
229
230     long registros = ventas.size();
231
232     double total =
233         ventas.stream()
234             .mapToDouble(VentaCafe::getDinero)
235             .sum();
236
237     double promedio =
238         ventas.stream()
239             .mapToDouble(VentaCafe::getDinero)
240             .average()
241             .orElse(0);
242
243     List<String> reporte = List.of(
244         "REPORTE DE VENTAS",
245         "-----",
246         "Total registros: " + registros,
247         "Total vendido: " + total,
248         "Promedio venta: " + promedio
249     );
250
251     Files.write(
252         Paths.get(first: "reporte_ventas.txt"),
253         reporte
254     );
255
256     System.out.println(

```

```

256     System.out.println(
257         x: "\nReporte generado correctamente."
258     );
259 }
260 public void obtenerVentasDia() {
261     System.out.println(x: "Ventas dia");
262     ventas.stream()
263         .filter(v -> v.getMomentoDia().equalsIgnoreCase(anotherString: "Morning"))
264         .limit(maxSize: 10)
265         .forEach(System.out::println);
266 }
267 public void obtenerVentasFranja() {
268     System.out.println(x: "Calcular venta por franja");
269     Map<String, Long> ventasPorMomento =
270         ventas.stream()
271             .collect(Collectors.groupingBy(
272                 VentaCafe::getMomentoDia,
273                 Collectors.counting()
274             ));
275
276     ventasPorMomento.forEach(
277         (momento, cantidad) ->
278             System.out.println(momento + ": " + cantidad)
279     );
280 }
281
282

```

Pantallas de Ejecución

Carga y exploración inicial

```

TOTAL REGISTROS: 3547

PRIMEROS 10 REGISTROS
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=12 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=12 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=13 | Cafe=Americano | Venta=28,90 | Pago=card | Momento=Afternoon
Hora=13 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=15 | Cafe=Americano with Milk | Venta=33,80 | Pago=card | Momento=Afternoon
Hora=16 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=18 | Cafe=Americano with Milk | Venta=33,80 | Pago=card | Momento=Night
Hora=19 | Cafe=Cocoa | Venta=38,70 | Pago=card | Momento=Night
Hora=19 | Cafe=Americano with Milk | Venta=33,80 | Pago=card | Momento=Night

```

Interfaces funcionales: Comparator, Consumer, Predicate, Function y BiFunction

```
COMPARATOR
Hora=11 | Cafe=Espresso | Venta=18,12 | Pago=card | Momento=Morning
Hora=9 | Cafe=Espresso | Venta=18,12 | Pago=card | Momento=Morning
Hora=14 | Cafe=Espresso | Venta=18,12 | Pago=card | Momento=Afternoon
Hora=21 | Cafe=Espresso | Venta=18,12 | Pago=card | Momento=Night
Hora=9 | Cafe=Espresso | Venta=18,12 | Pago=card | Momento=Morning
Hora=12 | Cafe=Espresso | Venta=18,12 | Pago=card | Momento=Afternoon
Hora=14 | Cafe=Espresso | Venta=18,12 | Pago=card | Momento=Afternoon
Hora=11 | Cafe=Espresso | Venta=18,12 | Pago=card | Momento=Morning
Hora=11 | Cafe=Espresso | Venta=18,12 | Pago=card | Momento=Morning
Hora=20 | Cafe=Espresso | Venta=18,12 | Pago=card | Momento=Night
```

```
CONSUMER
Latte -> $38.7
Hot Chocolate -> $38.7
Hot Chocolate -> $38.7
Americano -> $28.9
Latte -> $38.7
Americano with Milk -> $33.8
Hot Chocolate -> $38.7
Americano with Milk -> $33.8
Cocoa -> $38.7
Americano with Milk -> $33.8
```

```
PREDICATE
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=12 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=12 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=13 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=16 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=19 | Cafe=Cocoa | Venta=38,70 | Pago=card | Momento=Night
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=13 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=17 | Cafe=Cocoa | Venta=38,70 | Pago=card | Momento=Night
Hora=18 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Night
```

```
FUNCTION
Latte - $38.7
Hot Chocolate - $38.7
Hot Chocolate - $38.7
Americano - $28.9
Latte - $38.7
Americano with Milk - $33.8
Hot Chocolate - $38.7
Americano with Milk - $33.8
Cocoa - $38.7
Americano with Milk - $33.8
```

```
BIFUNCTION
Producto: Latte | Venta: $38.7
Producto: Hot Chocolate | Venta: $38.7
Producto: Hot Chocolate | Venta: $38.7
Producto: Americano | Venta: $28.9
Producto: Latte | Venta: $38.7
Producto: Americano with Milk | Venta: $33.8
Producto: Hot Chocolate | Venta: $38.7
Producto: Americano with Milk | Venta: $33.8
Producto: Cocoa | Venta: $38.7
Producto: Americano with Milk | Venta: $33.8
```

Cafés distintos, filtro de Latte y map a mayúsculas

```

CAFES DISTINTOS
Americano
Americano with Milk
Cappuccino
Cocoa
Cortado
Espresso
Hot Chocolate
Latte

SOLO LATTE
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=13 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=9 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=14 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=17 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Night
Hora=11 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=18 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Night

MAP
LATTE
HOT CHOCOLATE
AMERICANO
AMERICANO WITH MILK
COCOA
CORTADO
ESPRESSO
CAPPUCCINO

```

Promedio, estadísticas y top de cafés

```

PROMEDIO: 31.645215675218495

TOTAL VENDIDO: 112245.58
MAXIMO: 38.7
MINIMO: 18.12

TOP 10 CAFES
Americano with Milk=809
Latte=757
Americano=564
Cappuccino=486
Cortado=287
Hot Chocolate=276
Cocoa=239
Espresso=129

```

Reporte generado y agrupaciones por día, franja y jornada

```

Reporte generado correctamente.
Ventas dia
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=10 | Cafe=Americano | Venta=28,90 | Pago=card | Momento=Morning
Hora=10 | Cafe=Americano with Milk | Venta=33,80 | Pago=card | Momento=Morning
Hora=11 | Cafe=Americano with Milk | Venta=33,80 | Pago=card | Momento=Morning
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=11 | Cafe=Cortado | Venta=28,90 | Pago=card | Momento=Morning
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=10 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Hora=11 | Cafe=Americano with Milk | Venta=33,80 | Pago=card | Momento=Morning
Hora=9 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Morning
Calcular venta por franja
Afternoon: 1205
Night: 1161
Morning: 1181

VENTAS ORDENADAS POR JORNADA
Hora=12 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=12 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=13 | Cafe=Americano | Venta=28,90 | Pago=card | Momento=Afternoon
Hora=13 | Cafe=Latte | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=15 | Cafe=Americano with Milk | Venta=33,80 | Pago=card | Momento=Afternoon
Hora=16 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
Hora=14 | Cafe=Americano | Venta=28,90 | Pago=card | Momento=Afternoon
Hora=16 | Cafe=Americano with Milk | Venta=33,80 | Pago=card | Momento=Afternoon
Hora=12 | Cafe=Americano | Venta=28,90 | Pago=card | Momento=Afternoon
Hora=13 | Cafe=Hot Chocolate | Venta=38,70 | Pago=card | Momento=Afternoon
PS C:\Users\HP\Downloads\cafeteriacsv>

```

Explicación técnica de la solución

El programa lee el archivo Coffe_sales.csv (3.547 registros) y construye una lista de objetos VentaCafe con los datos de cada venta. El proyecto tiene tres clases: VentaCafe (el modelo), ServicioVentas (la lógica) y Main (donde se ejecuta todo).

Se usaron las interfaces funcionales de java.util.function: Comparator para ordenar por valor y por jornada, Consumer para mostrar café y valor, Predicate para filtrar ventas mayores a \$35, Function para describir cada venta y BiFunction para combinar café y valor en un texto de reporte. Con Streams se obtuvieron los cafés distintos, se filtraron los Latte, se pasaron nombres a mayúsculas y se calcularon promedio, total, máximo y mínimo. Con Collectors.groupingBy se agruparon las ventas por café y por franja horaria. Al final todo esto se guarda en reporte_ventas.txt

Conclusiones

Con los 3.547 registros analizados se vio que el promedio por venta fue de \$31,65 y el total vendido llegó a \$112.245,58. El café más vendido fue Americano with Milk (809 unidades), seguido de Latte (757) y Americano (564).

Trabajar con Streams e interfaces funcionales facilitó mucho el manejo de los datos: en lugar de recorrer listas con bucles y condicionales, cada operación (ordenar, filtrar, transformar, agrupar) quedó resuelta en pocas líneas y de forma más clara.